

# 11주차

## Word2Vec이란?

Word2Vec은 2013년 구글이 공개한 단어 임베딩 모델로, 단어 간 유사성을 수치로 표현하기 위한 모델이다.

핵심 기반 개념은 분포 가설(Distributional hypothesis).

### ✓ 분포 가설

| 같은 문맥에 자주 등장하는 단어는 비슷한 의미를 가진다.

예)

"자동차를 타고 부산에 간다"

"비행기를 타고 부산에 간다"

→ "자동차", "비행기"는 같은 문맥에 등장하므로 의미가 유사하다고 판단.

이러한 문맥적 유사성으로 단어를 벡터로 표현하는 것이 Word2Vec의 목표다.

## 희소 표현(Sparse Representation) vs 밀집 표현(Dense Representation)

### ✓ 희소 표현

- 원-핫 인코딩, TF-IDF처럼 대부분 0으로 채워진 매우 큰 벡터
- 단점:
  - 단어 사전 크기가 크면 벡터도 매우 큼 (비효율)
  - 단어 간 유사도를 계산하기 어려움 (0/1로만 구성)

### ✓ 밀집 표현

- Word2Vec이 학습해 만드는 작고 연속적인 값의 벡터
- 장점:

- 단어 간 의미적 관계를 벡터 공간에서 잘 표현
- 유사한 단어들은 비슷한 위치에 배치됨

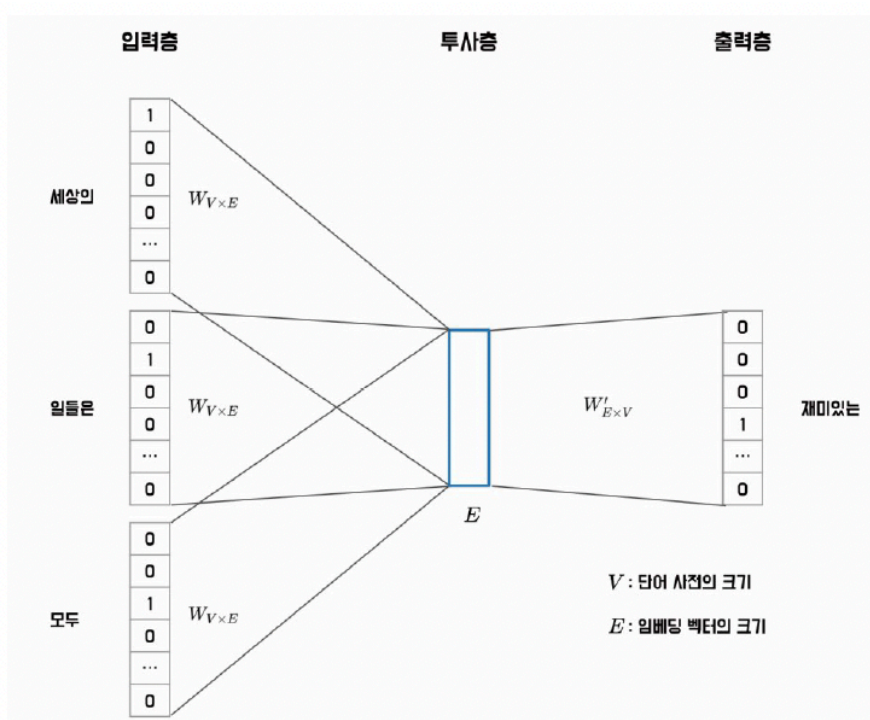
예)

- "자동차"와 "비행기" → 벡터 공간에서 가까움
- "고양이"와 "강아지" → 비슷한 영역

## Word2Vec의 두 가지 학습 방식

Word2Vec은 CBoW와 Skip-gram 두 종류로 학습된다.

### CBoW (Continuous Bag of Words)



#### ✓ 개념

주변 단어(Context words)로 중심 단어(Center word)를 예측하는 방식

문장에서 중심 단어 주변의 단어들을 입력으로 넣어 중심 단어를 맞힌다.

#### ✓ 학습 예시

입력 문장:

“세상의 재미있는 일들은 모두 밖에 일어난다”

윈도우 크기=2라면 학습 데이터는 다음과 같이 생성됨:

| 입력(주변 단어) | 출력(중심 단어) |
|-----------|-----------|
| 재미있는, 일들은 | 세상의       |
| 세상의, 일들은  | 재미있는      |
| 세상의, 재미있는 | 일들은       |
| ...       | ...       |

## ✓ 처리 과정

- 입력 단어 → 원-핫 벡터
- Lookup table(임베딩 행렬)에서 해당 단어의 임베딩 벡터 가져옴
- 주변 단어 임베딩들의 평균 벡터 계산
- 평균벡터  $\times W \rightarrow \text{Softmax} \rightarrow$  중심 단어 예측

## ✓ 특징

- 적은 데이터에서도 잘 학습됨
- 전체 문맥을 평균 내기 때문에 더 “부드러운” 결과

---

# Skip-gram

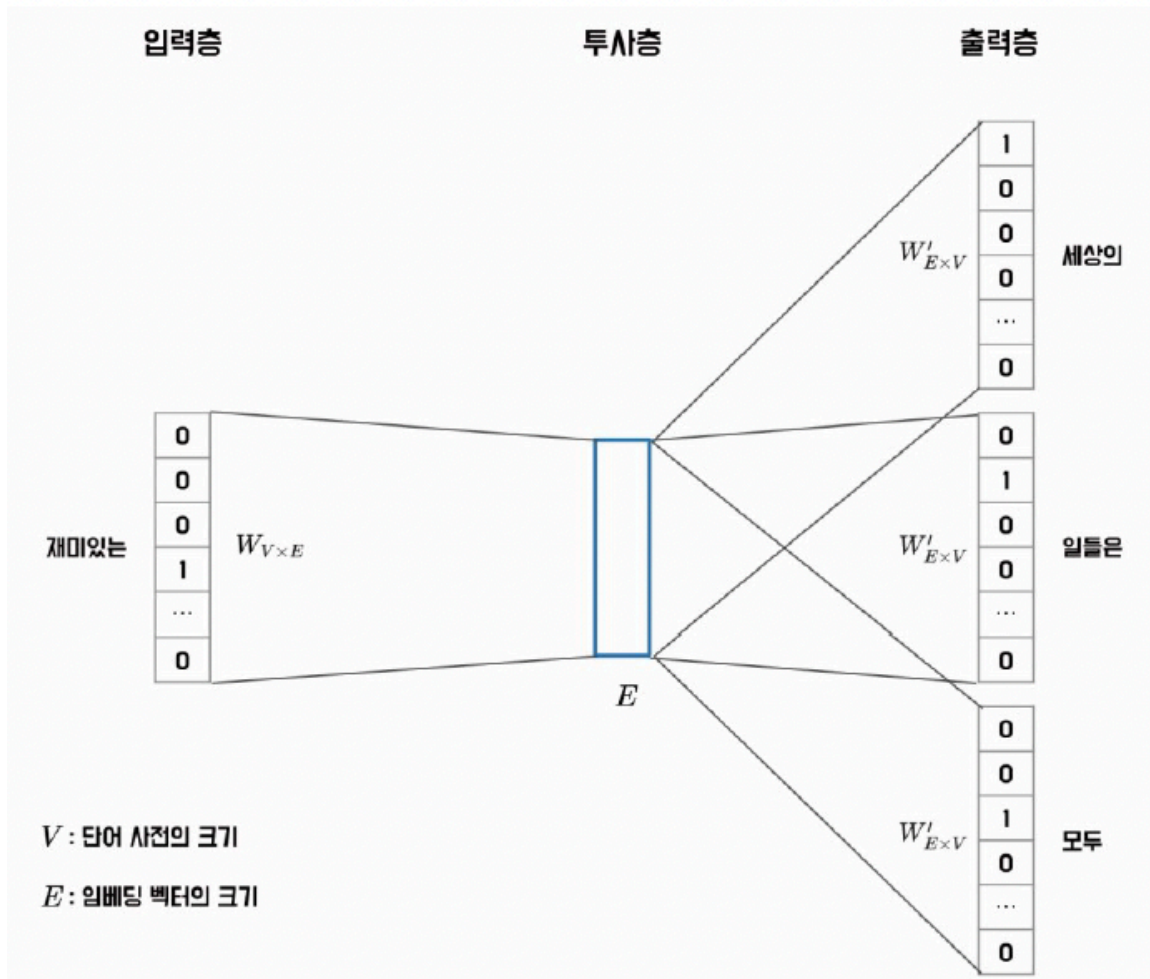


그림 6.7 Skip-gram 모델 구조

## ✓ 개념

중심 단어로 주변 단어들을 예측하는 방식

CBoW의 반대 구조.

## ✓ 학습 데이터 예시

| 입력(중심) | 출력(주변) |
|--------|--------|
| 재미있는   | 세상의    |
| 재미있는   | 일들은    |
| 재미있는   | 모두     |
| ...    | ...    |

## ✓ 특징

- 중심 단어를 기준으로 여러 개의 주변 단어를 예측하기 때문에  
더 큰 학습 데이터셋 생성 가능 → 대량 데이터에서 더 뛰어난 성능
  - 희귀 단어 학습에 강함
- 

## Word2Vec의 구조

Word2Vec은 단순한 신경망으로 구성됨:

입력층 → 투사층(Embedding layer) → 출력층

### ✓ 입력층

원-핫 벡터 형태

### ✓ 투사층 (Projection Layer)

- 실제 임베딩 벡터를 가져오는 Lookup Table
- 매개변수:
  - $(W_{\{V \times E\}})$  : 단어 사전  $V$ , 임베딩 차원  $E$

### ✓ 출력층

- Softmax를 통해 단어 확률 분포 예측
- 

## Word2Vec의 효율성 문제 & 해결 방법

Softmax는

- 단어 사전 크기 ( $V$ )가 크면 계산량이 매우 많음 → 느림

그래서 Word2Vec은 두 가지 최적화 기법을 사용:

---

## 계층적 소프트맥스(Hierarchical Softmax)

### ✓ 개념

Softmax 계산을 이진 트리(Binary tree) 구조로 대체하여 효율적으로 확률을 계산.

## 장점

- 시간 복잡도:
    - 기존 Softmax  $\rightarrow O(V)$
    - 계층적 Softmax  $\rightarrow O(\log V)$
  - 등장 빈도가 높은 단어를 트리 상위에 배치하여 효율적 예측 가능
- 

## 네거티브 샘플링(Negative Sampling)

### ✓ 개념

실제 단어(Positive) 몇 개 + 랜덤으로 뽑은 가짜 단어(Negative)를 사용해 학습  
다중 분류를 이진 분류로 단순화

예)

“재미있는”  $\rightarrow$  실제 주변 단어를 Positive(1)

그 외 랜덤 단어를 Negative(0)

## 장점

- 빠른 학습
  - 임베딩 품질도 우수
  - Word2Vec에서 가장 널리 쓰임
- 

## PyTorch 임베딩 클래스 설명

### ✓ nn.Embedding

Lookup table 구현을 위한 파이토치 클래스

핵심 파라미터:

- num\_embeddings : 단어 사전 크기(V)
- embedding\_dim : 임베딩 벡터 차원(E)
- padding\_idx : 패딩 단어 인덱스(해당 벡터는 학습 X)
- max\_norm : 임베딩 벡터 최대 크기 제한

- norm\_type : 정규화 방식(L2가 기본)

이 클래스를 사용하면 손쉽게 Word2Vec의 Lookup 기능을 구현할 수 있다.

## Skip-gram 모델 구현 예시(PyTorch)

```
class VanillaSkipgram(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(
            num_embeddings=vocab_size,
            embedding_dim=embedding_dim
        )
        self.linear = nn.Linear(
            in_features=embedding_dim,
            out_features=vocab_size
        )

    def forward(self, input_ids):
        embeddings = self.embedding(input_ids)
        output = self.linear(embeddings)
        return output
```

이 모델은:

- 중심 단어 임베딩 → Linear layer → 주변 단어 예측
- Softmax/Negative Sampling으로 학습

## Word2Vec 실습 과정

### 텍스트 전처리

#### ✓ 1) 데이터 로드

- NSMC 영화 리뷰 데이터셋(Korpora) 사용
- corpus.test에서 텍스트 로드

```
corpus = Korpora.load("nsmc")
corpus = pd.DataFrame(corpus.test)
```

## ✓ 2) 형태소 분석 (Okt)

```
tokenizer = Okt()
tokens = [tokenizer.morphs(review) for review in corpus.text]
```

## 결과

각 문장을 형태소(토큰) 단위로 분리한 리스트 생성됨.

# 단어 사전(Vocabulary) 구축

## ✓ 1) 단어 등장 횟수 세기

```
counter = Counter()
for sentence in corpus:
    counter.update(sentence)
```

## ✓ 2) 상위 n\_vocab개 단어만 선택

- 너무 많은 단어 사용 → 모델 크기 증가
- 상위 5,000개 단어만 vocab으로 사용

```
vocab = special_tokens + counter.most_common(n_vocab)
```

## ✓ 3) special token

- <unk>: 사전에 없는 단어를 처리하기 위한 토큰
- vocabulary 크기 = 5001

# Skip-gram용 학습 데이터 만들기

Skip-gram 목표:



| 중심 단어(Center word) → 주변 단어(Context word) 예측

## ✓ 1) 단어 쌍 생성 (get\_word\_pairs)

window\_size = 2

→ 중심 단어 기준 앞뒤 2개 단어까지 포함

예)

문장: 세상 의 재미있는 일 들 은 ...

단어쌍 예:

```
['재미있는', '세상']  
['재미있는', '의']  
['재미있는', '일']  
...
```

코드:

```
for idx, center in enumerate(sentence):  
    context = sentence>window_start:idx] + sentence[idx+1>window_end]
```

## ✓ 2) 단어쌍을 인덱스로 변환

Word2Vec은 단어가 아니라 정수 인덱스를 입력으로 사용한다.

```
center_index = token_to_id.get(center, unk_index)  
context_index = token_to_id.get(context, unk_index)
```

출력 예:

```
[[595, 100], [100, 595], ...]
```

# PyTorch용 Dataset 구성

## ✓ Tensor 변환

```
center_indexes = tensor(index_pairs[:,0])
context_indexes = tensor(index_pairs[:,1])
dataset = TensorDataset(center_indexes, context_indexes)
```

### ✓ DataLoader로 묶기

```
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

## Skip-gram 모델 구성 (PyTorch)

### ✓ VanillaSkipgram 클래스

구조:

입력 → 임베딩층 → 선형층 → 소프트맥스 → 단어 확률

임베딩층:

```
self.embedding = nn.Embedding(vocab_size, embedding_dim)
```

출력층:

```
self.linear = nn.Linear(embedding_dim, vocab_size)
```

### ✓ Forward

```
embeddings = self.embedding(input_ids)
output = self.linear(embeddings)
```

## 모델 학습

손실 함수: CrossEntropyLoss

최적화: SGD

```
for input_ids, target_ids in dataloader:
    logits = word2vec(input_ids)
    loss = criterion(logits, target_ids)
```

## 학습 결과

Loss가 epoch마다 감소 → 임베딩이 잘 학습됨.

## 학습된 임베딩 추출

Word2Vec 구조에서 임베딩 벡터는  
임베딩 층의 weight에 들어 있음.

```
embedding_matrix = word2vec.embedding.weight.detach().numpy()
```

단어별 embedding 저장:

```
for word, emb in zip(vocab, embedding_matrix):
    token_to_embedding[word] = emb
```

출력 예 (연기):

```
[-0.394 ... -0.654]
```

## 단어 유사도 계산 (Cosine Similarity)

### ✓ 코사인 유사도 식

[  
$$\cos(a,b) = \frac{a \cdot b}{||a|| \cdot ||b||}$$
  
]

### ✓ 단어 유사도 계산 코드

```
cosine = np.dot(a, b) / (norm(a) * norm(b))
```

## ✓ 가장 유사한 단어 찾기

```
closest = cosine_matrix.argsort()[::-1][:n]
```

출력 예:

“연기”와 가장 유사한 단어

- 연기력 (0.39)
- 배우 (0.31)
- 시나리오 (0.31)
- 악마 (0.29)
- 까지도 (0.28)

## Word2Vec Gensim 버전

PyTorch로 직접 학습하지 않고,

Gensim Word2Vec 클래스로 빠르게 모델 생성 가능

설치:

```
pip install gensim
```

## ✓ Gensim Word2Vec 핵심 파라미터

```
Word2Vec(  
    sentences,      # 토큰화된 문장  
    vector_size=100, # 임베딩 차원  
    window=5,      # 문맥 크기  
    min_count=5,    # 최소 등장 빈도  
    sg=1,          # 1: skip-gram, 0: CBoW  
    hs=0,          # hierarchical softmax 사용 여부  
    negative=5,     # negative sampling 개수  
    epochs=5  
)
```

## ✓ Gensim으로 학습 코드 예시

```
word2vec = Word2Vec(  
    sentences=tokens,  
    vector_size=128,  
    window=5,  
    min_count=1,  
    sg=1,  
    epochs=3  
)
```

학습 후 바로 사용 가능:

```
word2vec.wv['연기']  
word2vec.wv.most_similar('연기')
```

## ✓ Word2Vec의 목적

- 단어를 의미 기반 벡터로 표현하는 임베딩 모델
- 분포 가설 기반:  
비슷한 문맥에 등장하는 단어 → 비슷한 의미 → 비슷한 벡터

## ✓ 임베딩 방법의 두 종류

| 표현 방식          | 특징                              |
|----------------|---------------------------------|
| 희소 표현 (Sparse) | 원-핫 벡터, 거의 0으로 채워짐, 유사도 계산 어려움  |
| 밀집 표현 (Dense)  | Word2Vec이 생성, 작은 연속 값, 의미 기반 벡터 |

## ✓ Word2Vec의 두 가지 학습 방식

### 1) CBoW

- 주변 단어 → 중심 단어 예측
- 작은 데이터에서 잘 동작

### 2) Skip-gram

- 중심 단어 → 주변 단어 예측
  - 많은 학습 데이터에서 더 뛰어남
  - Word2Vec에서 가장 많이 사용됨
- 

## ✓ Word2Vec의 효율성 개선

### 1) 계층적 소프트맥스

- Softmax를 이진 트리로 대체
- 계산량  $O(\log V)$

### 2) 네거티브 샘플링

- 중심 단어와 진짜 주변 단어(1) + 랜덤 단어(0)
  - 이진 분류로 문제 단순화
  - Word2Vec 실전에서 가장 많이 사용
- 

## Word2Vec 실습 흐름 (PyTorch)

---

### ✓ 1) 데이터 전처리

- NSMC 데이터 불러오기
  - Okt 형태소 분석기로 토큰화
  - Sentence → token 리스트
- 

### ✓ 2) 단어 사전 구축

1. 단어 빈도 세기(Counter)
  2. 상위 5,000개 단어만 사용
  3. <unk>로 OOV 처리
  4. token\_to\_id 딕셔너리 생성
- 

### ✓ 3) Skip-gram 학습 데이터 생성

## 중심 단어 + 주변 단어 쌍 만들기

window\_size=2면:

```
[center="재미있는", context="세상"]  
[center="재미있는", context="의"]  
...
```

→ 단어쌍을 ID쌍으로 변환

## ✓ 4) DataLoader로 학습 데이터 구성

```
dataset = TensorDataset(center_indexes, context_indexes)  
dataloader = DataLoader(dataset, batch_size=32)
```

## ✓ 5) PyTorch Skip-gram 모델 구성

구조

입력 → Embedding → Linear → Softmax

Forward

```
emb = embedding(center_ids)  
logits = linear(emb)
```

## ✓ 6) 모델 학습

- Loss: CrossEntropyLoss
- Optimizer: SGD
- Epoch마다 loss 감소 → 임베딩 학습 성공

## ✓ 7) 임베딩 벡터 추출

```
embedding_matrix = model.embedding.weight.detach().numpy()
```

## ✓ 8) 단어 유사도 계산

### 코사인 유사도

```
[  
cos(a,b)=\frac{a\cdot b}{|a||b|}  
]
```

가장 유사한 단어 Top-5 출력 가능.

## Gensim Word2Vec 실습

Gensim은 Word2Vec을 빠르게 구현할 수 있는 라이브러리

### 주요 파라미터

```
Word2Vec(  
    sentences=tokens,  
    vector_size=128,  
    window=5,  
    min_count=5,  
    sg=1,      # skip-gram  
    negative=5,  
    epochs=3  
)
```

### 학습 후

```
wv['단어']  
wv.most_similar('단어')  
wv.similarity('단어A', '단어B')
```



# Word2Vec 한계와 FastText 등장

Word2Vec 한계:

- 단어를 "하나의 토큰"으로만 봄
- 형태소 복잡한 언어(한국어 등)에서 OOV 문제 심각
  - 예: "사랑해요"가 vocab에 없으면 임베딩 불가능

## FastText 핵심 정리

### ✓ FastText의 핵심 아이디어

단어를 자극 단위(subword)로 분해 = n-gram 방식

예: "사랑해요"

- <사
- 사랑
- 랑해
- 해요>

→ 여러 subword를 합쳐 단어 임베딩 구성

→ 단어가 처음 보더라도 하위 단어(subword) 조합으로 임베딩 생성 가능

👉 OOV 문제 해결

### ✓ FastText 학습 방식

- 단어 → n-gram → n-gram 임베딩들의 합
- Skip-gram/CBoW + Negative Sampling 사용
- Gensim에서 Word2Vec과 사용법 거의 동일

### ✓ FastText Gensim 사용 예

```
fastText = FastText(  
    sentences=tokens,  
    vector_size=128,
```

```

window=5,
min_count=5,
sg=1,
min_n=2,
max_n=6,
epochs=3
)

```

## OOV 단어 처리

Word2Vec: 불가

FastText: 가능 (subword 분해)

## RNN (Recurrent Neural Network)

Word2Vec/fastText가 단어 임베딩을 만드는 모델이라면,

RNN은 순서 있는 데이터를 처리하는 신경망 구조.

### ✓ RNN이 필요한 이유

자연어, 시계열 데이터는 순서·문맥 의존성을 갖는다.

이전 단어가 다음 단어 의미에 영향을 줌.

예)

“금요일이 지나면” → “주말”, “토요일” 예상됨

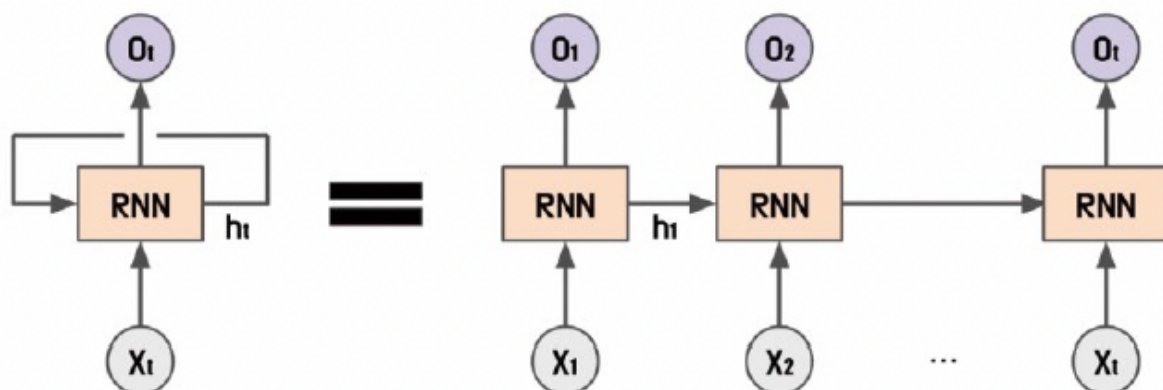


그림 6.12 순환 신경망의 셀

---

## ✓ RNN의 핵심 개념: 은닉 상태 (Hidden State)

RNN은 매 시점  $t$ 에서:

[

$$h_t = f(h_{t-1}, x_t)$$

]

- ( $h_{t-1}$ ): 이전 상태
- ( $x_t$ ): 현재 입력
- → 새로운 hidden state ( $h_t$ ) 계산
- hidden state를 통해 문맥 기억

---

## ✓ RNN의 출력 계산

[

$$y_t = g(W h_t + b)$$

]

---

# RNN의 구조 타입

## ✓ 1) 일대다 구조 (One-to-Many)

입력: 하나

출력: 시퀀스 전체

예: Image Captioning

- 이미지 한 장 입력 → 문장 생성

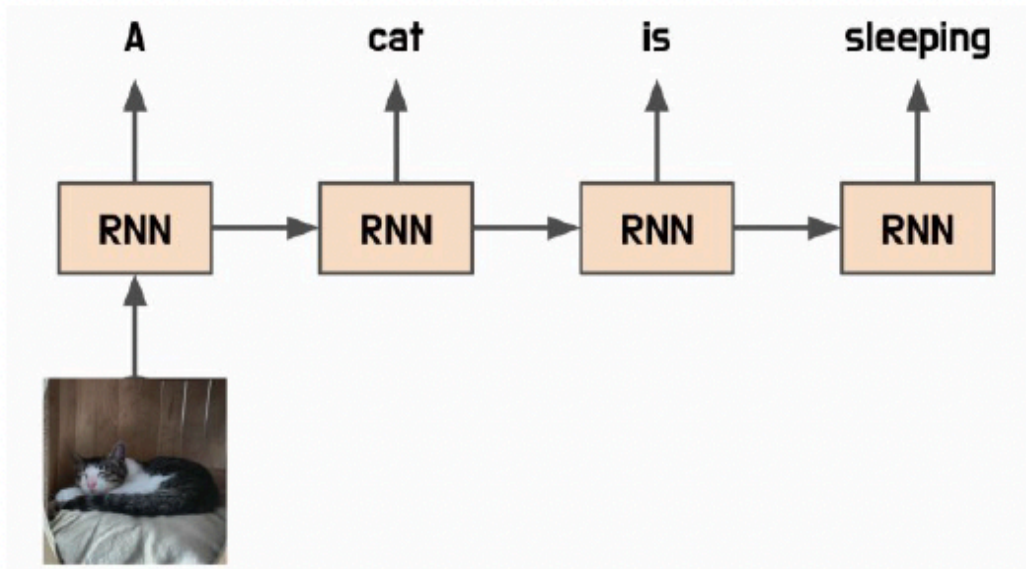


그림 6.13 순환 신경망의 일대다 구조

## ✓ 2) 다대일 구조 (Many-to-One)

입력: 문장 전체

출력: 하나의 라벨

예: 감정 분석

- 문장 → 긍정/부정

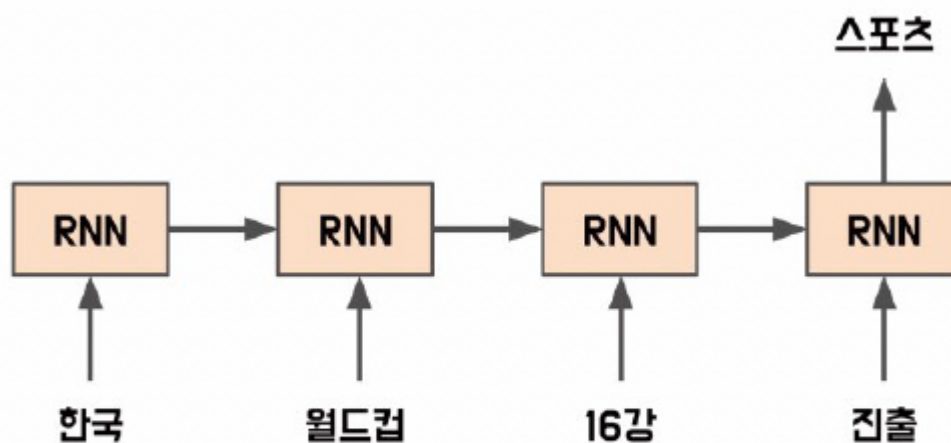


그림 6.14 순환 신경망의 다대일 구조

## 분포 가설 (Distributional Hypothesis)

- 같은 문맥에서 자주 등장하는 단어는 비슷한 의미를 가진다.
  - Word2Vec·fastText 등 대부분의 임베딩 모델의 근간.
- 

## 단어 표현 방식

### 1) 희소 표현 (Sparse Representation)

- 원-핫 인코딩, TF-IDF 등
- 대부분 0으로 이루어진 긴 벡터 → 비효율적, 의미 정보 없음

### 2) 밀집 표현 (Dense Representation)

- 실수값을 가진 짧은 벡터
  - 단어 의미를 공간상 위치로 표현
  - Word2Vec, fastText가 대표적
- 

## Word2Vec

Google(2013)에서 발표한 단어 임베딩 모델.

### 모델 구조

#### 1. CBoW

- 주변 단어 → 중심 단어 예측
- 빠르고 작은 데이터에 유리

#### 1. Skip-gram

- 중심 단어 → 주변 단어 예측
- 드문 단어 학습에 강함, 성능 더 좋음

### 문제점

- 단어를 "단위"로만 본다 → 한국어처럼 형태소 구조가 복잡한 언어에 불리
  - 사전에 없는 단어(OOV)를 처리 못함
- 

## 계층적 소프트맥스 (Hierarchical Softmax)

- 일반 Softmax의  $O(V)$  복잡도를  
→  $O(\log V)$  로 줄이기 위한 기법
  - 단어를 트리 구조로 구성하여 경로 확률로 계산
- 

## 네거티브 샘플링 (Negative Sampling)

- Softmax 대신 “정답 단어는 1, 랜덤 단어는 0”으로 학습
  - 빠르고 효과적, Skip-gram과 같이 많이 사용됨
- 

## Gensim Word2Vec

파이썬에서 Word2Vec을 쉽게 사용하는 라이브러리

중요 파라미터:

- vector\_size: 임베딩 차원
- window: 문맥 창 크기
- min\_count: 최소 등장
- sg: 1=Skip-gram, 0=CBoW
- negative: 네거티브 샘플링
- epochs: 학습 반복

ww.most\_similar(), ww.similarity() 등을 제공해 단어 유사도 계산 가능.

---

## fastText

Meta(FAIR, 2015)에서 개발한 단어 임베딩 모델.

### Word2Vec보다 개선된 점

- 단어를 문자 N-gram의 조합(하위 단어) 으로 본다  
예: “사랑해요” → ‘사’, ‘랑’, ‘해’, ‘요’, ‘사랑’, ‘랑해’, ...
- 장점:
  - OOV(사전에 없는 단어)도 처리 가능
  - 한국어·독일어처럼 형태가 복잡한 언어에 강함
  - 드문 단어도 잘 학습됨

---

## RNN (Recurrent Neural Network)

연속된 시퀀스 데이터를 처리하는 신경망 구조.

### 핵심 개념

- 입력 + 이전 은닉상태 → 현재 은닉상태 계산
- 시계열 의존성 반영

### 문제점

- 오래된 정보를 유지하기 어려움 (장기 의존성 문제)
- 역전파 중 기울기 소실/폭발 문제

---

## RNN 구조의 확장

### 1) 일대다 (One-to-Many)

문장 생성, 이미지 캡셔닝 등

입력 1 → 출력 여러 개

### 2) 다대일 (Many-to-One)

감정 분석, 문장 분류 등

문장 전체 입력 → 라벨 1개 출력

### 3) 다대다 (Many-to-Many)

번역(Seq2Seq), 음성 인식

입력 시퀀스 ↔ 출력 시퀀스

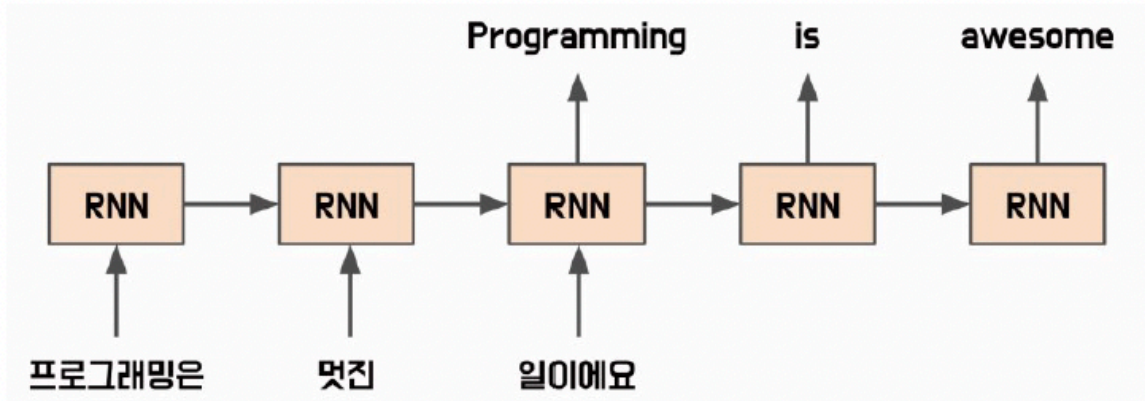


그림 6.15 순환 신경망의 다대다 구조

#### 4) 양방향 RNN (Bi-RNN)

앞에서 오는 정보 + 뒤에서 오는 정보를 모두 사용

정확도 상승

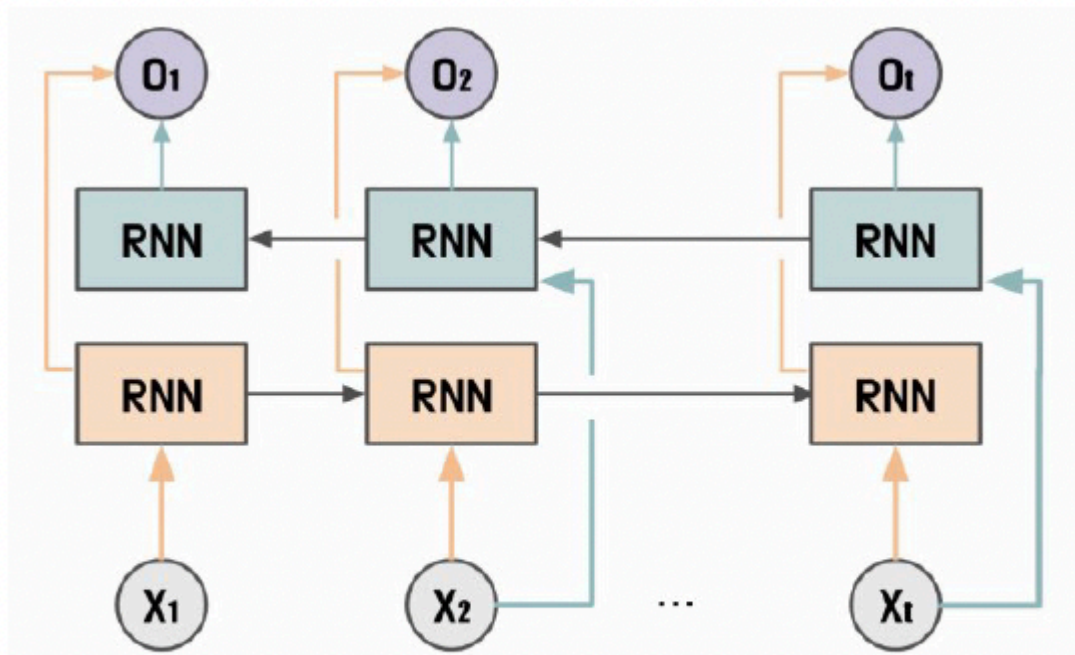


그림 6.16 양방향 순환 신경망 구조

#### 5) 다중 RNN (Stacked RNN)

여러 층을 쌓아 깊은 패턴 학습 가능



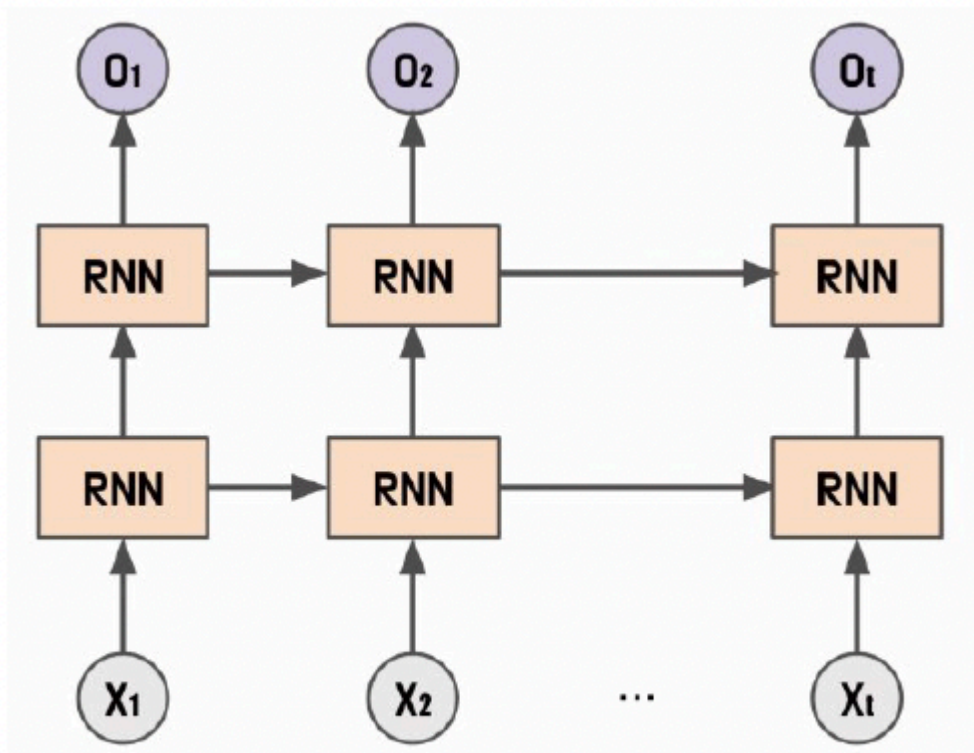


그림 6.17 다중 순환 신경망 구조

## LSTM (Long Short-Term Memory)

장기 의존성 문제를 해결하기 위해 고안된 RNN 계열 모델.

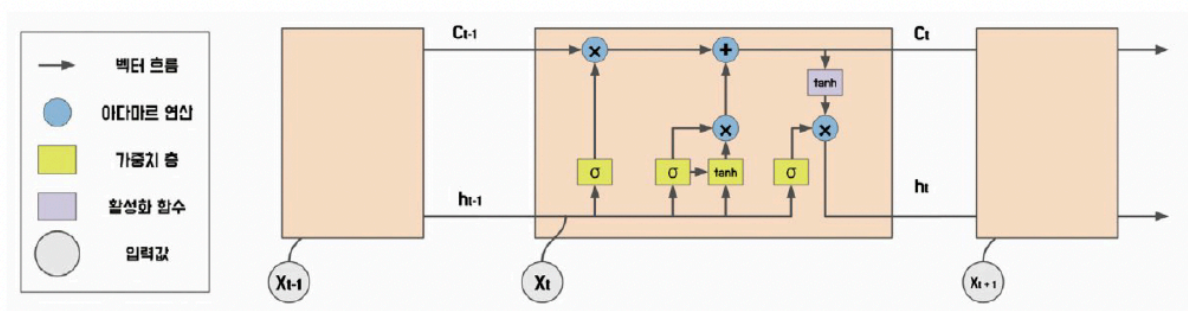


그림 6.18 장단기 메모리 구조

## 핵심 구성

LSTM은 “메모리 셀”을 가지고 있으며 3개의 게이트로 정보 흐름을 조절한다.

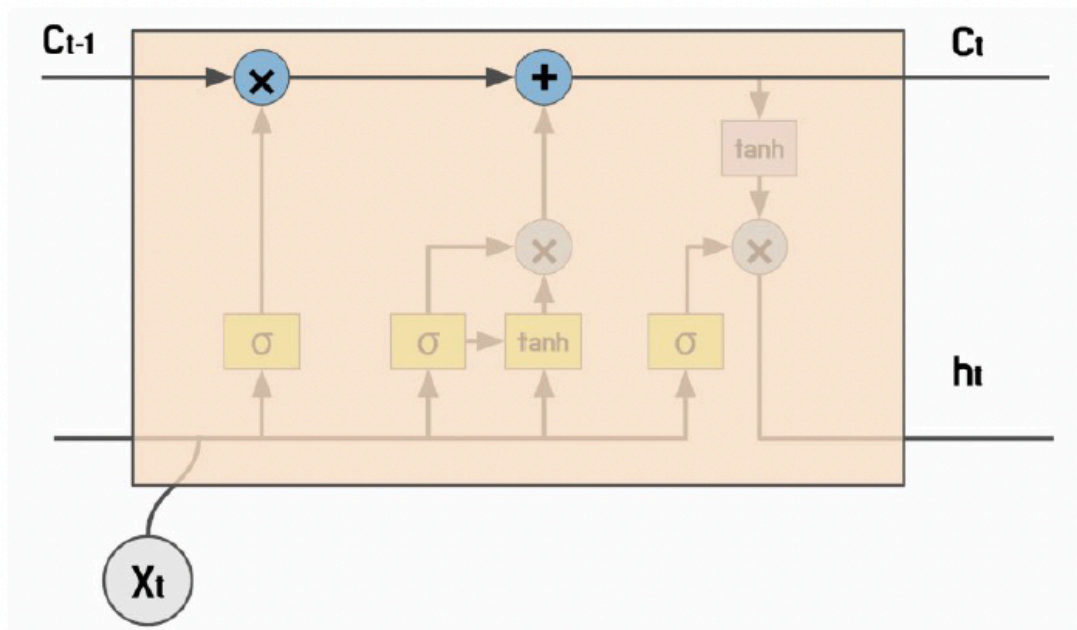


그림 6.19 장단기 메모리 셀 상태

### 1) Forget Gate (망각 게이트)

- 과거 정보 중 어떤 것을 버릴지 결정
- $\sigma \rightarrow 0 \sim 1$

### 2) Input Gate (입력 게이트)

- 어떤 새로운 정보를 메모리에 추가할지 결정
- $\sigma$ 와  $\tanh$  사용

### 3) Output Gate (출력 게이트)

- 현재 시점의 은닉 상태에 어떤 정보를 보낼지 결정

### 셀 상태(Cell state)

- RNN의 은닉 상태보다 더 길게 정보를 저장하는 통로
- 장기 기억 담당

LSTM 덕분에:

- 장기 의존성 학습 가능
- 긴 문장·문맥 기반 작업 효과 ↑

## ✓ Word2Vec (Skip-gram / CBOW)

단어를 고정된 길이의 벡터로 변환하는 알고리즘.

### ● 특징

- 단어 간 의미적 유사도를 반영
  - CBOW: 주변 단어 → 중심 단어 예측
  - Skip-gram: 중심 단어 → 주변 단어 예측
  - 단어 단위로만 학습 → 형태소 변화 반영 불가(OOV 문제)  
예: 연기하다, 연기력 → Word2Vec은 둘의 관계를 모름
- 

## FastText

Word2Vec의 단점 해결을 위해 Subword(하위 단어) 기반으로 임베딩 학습.

### ✓ 핵심 아이디어

단어를 N-gram 형태로 쪼갠 뒤 벡터를 학습

예: "사랑해요"

→ 사, 랑, 해, 요 + 사랑, 랑해, 해요 ...

### ✓ 장점

- OOV(사전에 없는 단어)도 처리 가능 → Subword 합성해서 벡터 생성
  - 한국어처럼 글자 단위/형태소 중요 언어에서 매우 유용
  - Word2Vec보다 일반적으로 더 성능 좋음
- 

## 순환 신경망 RNN

시퀀스(연속된 데이터)를 처리하는 신경망.

### ✓ 특징

- 이전 시점의 hidden state ( $h_{t-1}$ ) 를 이용 → “과거 정보 기억”
- 수식  
[

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$$

## ✓ 한계

- 긴 문장에서 과거 정보를 잊는 장기 의존성 문제
- 기울기 소실 문제(Vanishing gradient)

## RNN 구조 확장

### (1) 일대다 구조 (One-to-Many)

입력 1 → 출력 여러 개

ex) 이미지 → 캡션 생성 (Image Captioning)

### (2) 다대일 구조 (Many-to-One)

입력 여러 개 → 출력 1

ex) 감성 분석, 문장 분류

### (3) 다대다 구조 (Many-to-Many)

입력 여러 개 → 출력 여러 개

ex) 번역, 음성 인식

## 양방향 RNN (BiRNN)

현재 단어의 의미는 앞뒤 단어 모두 영향을 준다

→ RNN을 앞 방향 + 뒤 방향 두 개 사용

장점

- 문맥 이해↑
- 번역, 감성 분석 등에서 성능 우수

## 다중(Stacked) RNN

RNN 여러 층을 쌓음 → 더 복잡한 패턴 학습 가능

단점: 연산량 증가, 기울기 소실 가능성 증가

---

## LSTM (Long Short-Term Memory)

RNN의 장기 의존성 문제 해결 모델.

### ✓ 핵심 요소: 메모리 셀(Cell) + 세 가지 게이트

#### 1. Forget gate ( $f_t$ )

이전 기억을 얼마나 유지할지 결정

$$[f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)]$$

#### 2. Input gate ( $i_t$ ) + 후보 메모리 ( $g_t$ )

어떤 새로운 정보를 저장할지 결정

$$[i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)]$$

$$[g_t = \tanh(W_g [h_{t-1}, x_t] + b_g)]$$

#### 3. Cell state 업데이트

$$[c_t = f_t \odot c_{t-1} + i_t \odot g_t]$$

#### 4. Output gate ( $o_t$ )

어떤 정보를 출력으로 보낼지

$$[o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)]$$

#### 5. hidden state 계산

$$[h_t = o_t \odot \tanh(c_t)]$$

### ✓ 장점

- 장기 기억 가능
- 기울기 소실 문제 해결
- 텍스트 분류/번역 등 핵심 모델

## PyTorch의 RNN / LSTM 클래스

### ✓ RNN 클래스

- input\_size: 임베딩 차원
- hidden\_size: hidden state 크기
- num\_layers: 층 수
- batch\_first=True → 입력 shape이 (batch, seq, feature)
- bidirectional=True → 양방향 RNN

## ✓ LSTM 클래스

RNN과 동일 +

→ hidden state (h\_n)

→ memory cell (c\_n)

두 개 반환

또한 proj\_size 옵션 → LSTM 출력에 선형 투사 적용 가능

# 텍스트 분류 모델 전체 파이프라인

## ✓ (1) 데이터 불러오기 (Korpora NSMC 등)

## ✓ (2) 토큰화 & 단어 사전 구축

- Okt 형태소 분석기 사용
- 상위 n개 단어만 선택 (예: 5000개)
- <pad>, <unk> 토큰 추가

## ✓ (3) 정수 인코딩

문장을 단어 ID로 변환

ex) ['영화', '너무', '좋다'] → [120, 381, 42]

## ✓ (4) 패딩 (pad\_sequences)

모든 문장 길이를 동일하게 맞춤 (예: max\_length=32)

## ✓ (5) PyTorch TensorDataset & DataLoader 구성

## ✓ (6) 모델 정의 (SentenceClassifier)

- Embedding layer
- RNN 또는 LSTM 선택
- 양방향 여부 선택
- Dropout
- 마지막 hidden state → Linear → Logits 출력

## ✓ (7) 손실함수 & Optimizer

- Loss: BCEWithLogitsLoss (binary classification)
  - Optimizer: Adam
- 

# 자연어 처리 입력 파이프라인

## ✓ 코퍼스 로드 → 토큰화 → 단어 사전 구축 → 정수 인코딩 → 패딩 → DataLoader

### ① 토큰화(Okta 등)

문장을 형태소 단위로 나누기

### ② 단어 사전 만들기

가장 많이 등장하는 단어 top-k 선정

<pad>, <unk> 포함

### ③ 정수 인코딩

단어를 id로 변환 (모델 입력을 위해 숫자로 바꿈)

### ④ 패딩

문장의 길이를 동일하게 맞추기

짧으면 0()으로 채움

### ⑤ DataLoader

파이토치에서 배치 단위로 효율적 학습 수행

---

# RNN(순환 신경망)

## ✓ RNN 기본 구조

- 입력  $x_t$  + 이전 hidden  $h_{t-1}$  을 이용해 현재 hidden  $h_t$  계산
  - 장점:
    - 순차 데이터 처리
    - 문맥 반영
  - 단점:
    - 장기 의존성(Long-term dependency) 문제
    - 기울기 소실, 폭주 문제 발생
- 

## ✓ RNN의 세 가지 구조 형태

### 1. 일대다 (One-to-Many)

입력 하나 → 출력 여러 개

예: 이미지 → 캡션 생성(Image Captioning)

### 2. 다대일 (Many-to-One)

여러 단어 입력 → 감정 하나 출력

예: 감정 분석(Sentiment Analysis)

### 3. 다대다 (Many-to-Many)

입력: 문장

출력: 번역된 문장

예: 기계 번역(Seq2Seq)

---

## 양방향 RNN (BiRNN)

- 이전( $t-1$ ) 정보 + 이후( $t+1$ ) 정보를 모두 활용
  - 장점: 문맥 이해력 향상
  - 단점: 계산량 증가
-



## 다층 RNN (Stacked RNN)

- 여러 층의 RNN을 연결
- 깊은 패턴 학습 가능
- 단점: 계산량 증가, 학습 어려움

## LSTM (장단기 메모리)

RNN의 장기 의존성 문제를 해결하기 위한 구조.

### ✓ LSTM 구성 요소(핵심)

1. 망각 게이트 (Forget Gate)
  - 과거 정보를 얼마나 버릴지 결정
  - $f = \text{sigmoid}(Wx + Wh + b)$
2. 기억 게이트 (Input Gate + 후보 메모리)
  - $i = \text{sigmoid}(\dots)$
  - $g = \tanh(\dots)$
  - 새로운 정보 중 얼마나 저장할지 결정
  - 메모리 업데이트:  $c_t = f \odot c_{(t-1)} + i \odot g$
3. 출력 게이트 (Output Gate)
  - 현재 메모리에서 얼마나 출력할지 결정
  - $h_t = o \odot \tanh(c_t)$

LSTM은  $c_t$ (셀 상태) 를 통해 장기 정보를 안전하게 유지.

## 구현 요소 (PyTorch)

### ✓ RNN / LSTM 클래스

```
nn.RNN(input_size, hidden_size, num_layers, ...)
nn.LSTM(input_size, hidden_size, num_layers, ...)
```

## ✓ 출력값

- output: 모든 시점의 hidden state
- h\_n: 마지막 hidden state
- c\_n(LSTM만 해당): 마지막 cell state

## 문장 분류 모델 구조

입력 문장

- 임베딩 층
- RNN/LSTM
- 마지막 hidden state
- Dropout
- Linear classifier
- Sigmoid (이진 분류)

Loss: BCEWithLogitsLoss

Optimizer: RMSProp

## 사전 학습 임베딩 적용 (Word2Vec 사용)

### ✓ 학습한 Word2Vec 모델 불러오기 → 임베딩 초기화

- <pad> 와 <unk> 제외
- from\_pretrained 사용
- 임베딩 초기화하면 더 빠르게 수렴하거나 성능 향상 가능
- 단, 항상 좋은 것은 아님(데이터/언어 특성에 따라 다름)

## 합성곱 신경망 (CNN)

문장 분류나 이미지 처리에 모두 사용 가능.

### ✓ CNN 핵심 구성

#### 1. 합성곱 계층(Convolution Layer)

- 지역 특징(Local Feature) 추출
- 필터(Kernel)가 입력을 움직이며 연산 수행
- 여러 필터 → 여러 특징맵 생성

## 2. 필터(Kernel)

- 예: 3×3, 5×5
- 입력 이미지의 일부 영역과 곱셈 → 합산 → 특징값 추출

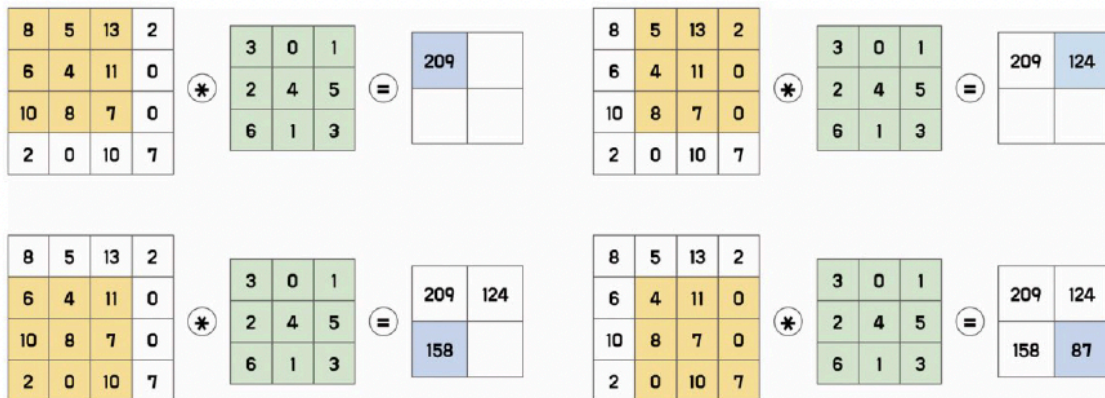


그림 6.24 특징 맵 추출 과정

## 3. 패딩(Padding)

- 출력 크기를 유지하기 위해 입력 주변에 0을 채움

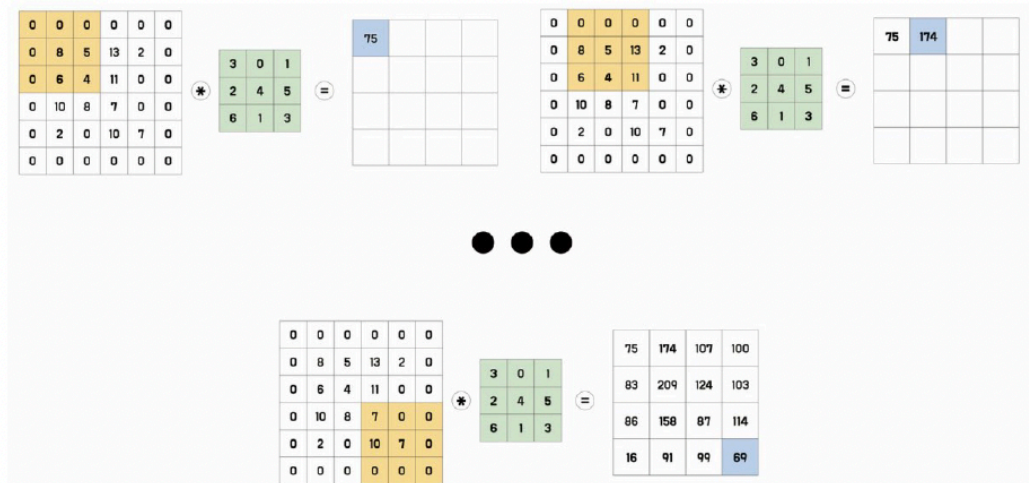


그림 6.25 제로 패딩

#### 4. 스트라이드(Stride)

- 필터가 움직이는 간격
- 클수록 출력 크기 감소, 계산량 감소

#### 5. 채널(Channel)

- 입력 이미지의 차원(RGB → 3 채널)
- 필터도 동일한 채널 수를 가짐

#### 6. 팽창(Dilation)

- 필터 사이에 간격을 두어 더 넓은 영역을 보게 하는 기법
- 시퀀스나 이미지의 장거리 패턴 학습 가능

## CNN(합성곱 신경망)

### ◆ CNN 특징

- 지역 특징(Local features) 잘 추출
- 가중치 공유로 파라미터 감소
- 이미지·자연어 모두 사용 가능

# 합성곱 계층(Conv2D) 핵심 요소

## ✓ 필터(kernel)

- 3×3, 5×5 형태가 일반적
- 필터가 입력 데이터의 일부 영역을 보면서 특징을 추출

## ✓ 패딩(Padding)

- 크기를 유지하기 위해 입력 주변에 0을 채움
- Zero-padding = 주변 0 추가

## ✓ 스트라이드(Stride)

- 필터 이동 간격
- 크면 특징맵이 작아짐

## ✓ 채널(Channel)

- 입력/출력 특징맵의 깊이 의미
- R,G,B → 채널 3
- 필터 개수 = 출력 채널 수

## ✓ 팽창(Dilation)

- 필터 원소 사이 간격 늘려 넓은 영역을 한 번에 보기

---

## 합성곱 출력 크기 공식

$$L_{out} = \lfloor \frac{L_{in} + 2p - d(k-1) - 1}{s} + 1 \rfloor$$

$$L_{out} = \lfloor \frac{L_{in} + 2p - d(k-1) - 1}{s} + 1 \rfloor$$

---

## 활성화 맵 (Activation Map)

- Conv 결과에 비선형성(ReLU 등)을 적용한 것
  - 모델이 더 복잡한 패턴 학습 가능
-

# 풀링(Pooling)

## ✓ MaxPooling

- 가장 큰 값을 선택

## ✓ AvgPooling

- 평균값을 선택

효과:

- 크기 감소 → 연산량 감소
- 위치 변동에 강함(translation invariance)

# 완전 연결 계층(FC)

- Flatten → 벡터로 변환 → Linear Layer
- 최종 분류 수행

# CNN 전체 구조 예시

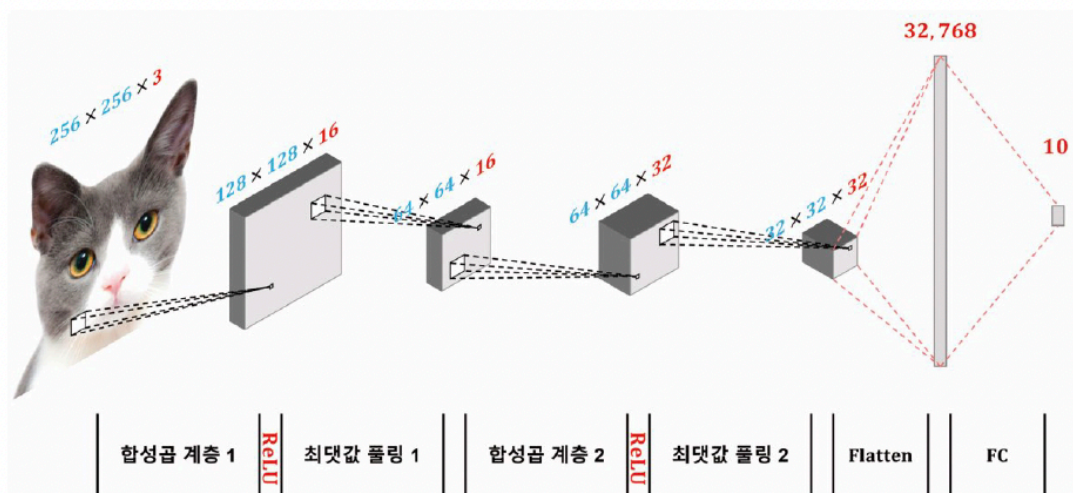


그림 6.28 합성곱 모델 구조 시각화

입력 이미지를 다음 순서로 처리:

Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → FC

PyTorch 구현 예제 포함됨.

## 왜 텍스트에 2D 합성곱이 안 되는가?

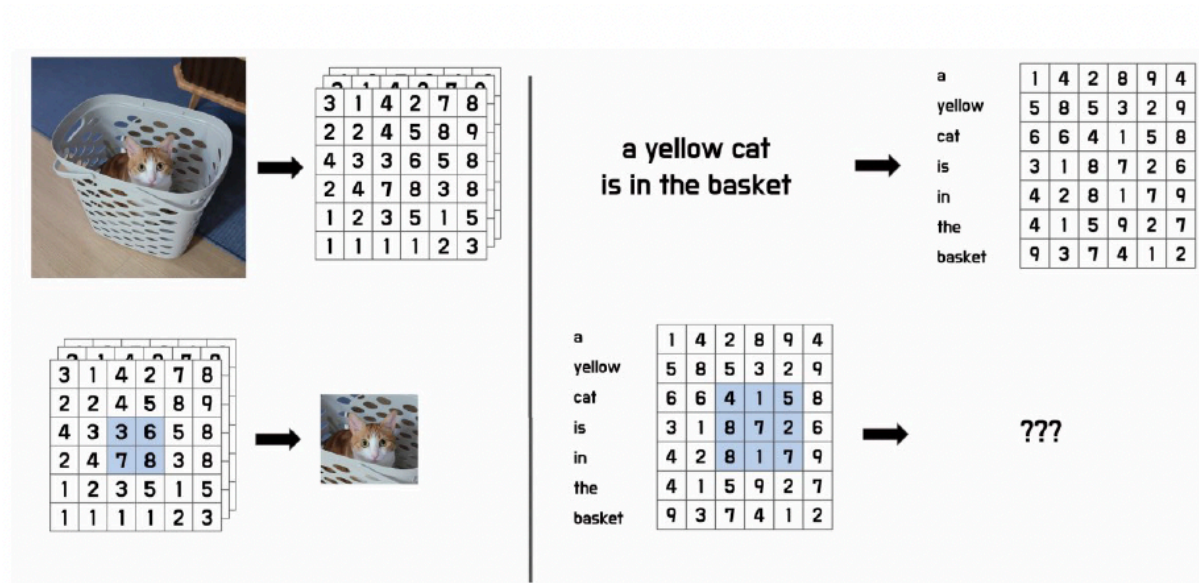


그림 6.29 이미지 데이터와 텍스트 데이터의 2차원 합성곱

### ✓ 이미지 데이터

- (높이 × 너비 × 채널) 구조
- 위치 정보가 매우 중요
- 픽셀의 상하좌우 배열·형태가 의미를 가짐 → 2D 합성곱 적합

### ✓ 텍스트 데이터

- 단어 임베딩을 수직 방향으로 쌓아 만든 2D 형태처럼 보이나 실제로는 위치 정보가 오직 수평 방향(문장 흐름)에만 의미가 있음.
- 단어 임베딩 행(row) 간의 상하관계는 의미가 없음 → 2D 합성곱 불가능

➡ 결론: 텍스트에는 1차원 합성곱(1D Convolution)을 사용해야 한다.

# 1차원 합성곱(1D CNN)의 개념

문장을 임베딩하면 다음과 같은 행렬 형태가 된다:

문장 길이 x 임베딩 차원  
(sequence\_length × embedding\_dim)

## ✓ 1D 합성곱 방식

- 필터 크기(height): 단어 개수 (예: 2, 3, 4-gram)
- 필터 길이(width): embedding\_dim과 동일 (전체 벡터 폭을 덮음)
- 이동 방향: 수평 방향(문장 순서)

즉, 한 번에 N개 단어를 묶어서 특징 추출 → CNN 기반 n-gram 효과

## 1D 합성곱 연산 예시

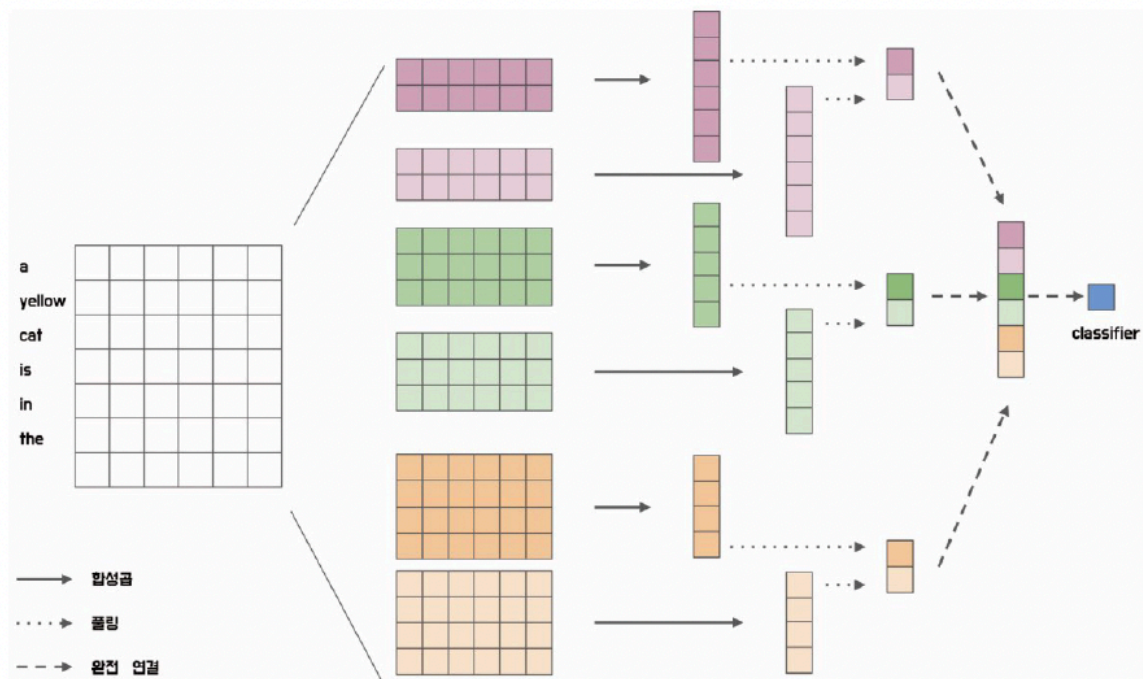


그림 6.31 1차원 합성곱을 이용한 출력 벡터 계산

예: 필터 크기=3

→ a yellow cat 세 단어에서 특징 추출



→ 다음 위치 yellow cat is

→ 계속 오른쪽으로 이동하며 각 위치에서 스칼라 값을 추출

이 값들이 합쳐져 1차원 특징 벡터가 된다.

---

## 다양한 크기의 필터 사용 (핵심!)

CNN 기반 텍스트 분류는 보통 다음 필터 크기를 사용한다:

- 2 (bi-gram)
- 3 (tri-gram)
- 4
- 5

각 필터는 서로 다른 문맥적 특징을 포착한다.

따라서 여러 필터를 동시에 적용하면:

- ✓ 다양한 크기의 n-gram 패턴 학습
- ✓ 각 필터에서 1개의 스칼라 특징을 출력
- ✓ 모든 필터 결과를 concat → 하나의 큰 feature vector

이게 텍스트 CNN 분류기의 핵심 구조다.

---

## 풀링(Pooling)의 역할

1D CNN에서는 MaxPooling(전체 pooling) 을 사용한다.

### ✓ 이유

- 필터가 출력한 스칼라 리스트에서 가장 중요한 특징(가장 큰 값) 만 선택
- 전체 문장에서 그 특징의 "존재 여부"를 반영
- n-gram 위치에 상관 없이 전역 특징을 유지

예:

yellow cat is ... 중에서 "긍정 표현" 특징이 한 번이라도 강하게 나타났다면 → positive

---

## 1D CNN 전체 구조

텍스트를 위한 CNN 모델 구조는 다음과 같다:

문장 → 임베딩 → 다중 크기 Conv1D → ReLU → MaxPooling → concat → Fully Connected → 출력

각 부분 설명:

## Embedding

문장을 (sequence\_length × embedding\_dim)의 텐서로 변환

## 여러 크기의 1D Conv

- 필터 크기 = 2, 3, 4, 5 ...
- 출력 채널 = 보통 필터당 1

## ReLU

비선형성 적용

## MaxPooling

각 필터가 낸 출력 중 최대값 추출 → 스칼라 1개

## Concat

모든 필터로 얻은 스칼라들을 연결 → 하나의 큰 벡터

## Fully Connected

최종 분류 수행

---

## 사전 학습 임베딩 사용(Word2Vec)

텍스트 CNN에서는 사전 학습 임베딩을 사용하는 것이 일반적이며 성능 향상에 도움됨.

- Word2Vec, GloVe 등에서 학습된 벡터 사용
- nn.Embedding.from\_pretrained 로 초기화
- 미세 조정(finetune) 가능

---

## 모델 코드 핵심

## 모델 정의 (여러 필터 사이즈 적용)

```
conv = []
for size in filter_sizes:
    conv.append(
        nn.Sequential(
            nn.Conv1d(in_channels=embedding_dim, out_channels=1, kernel_size=size),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=max_length - size + 1)
        )
    )
self.conv_filters = nn.ModuleList(conv)
```

## Forward

```
embeddings = self.embedding(inputs)
embeddings = embeddings.permute(0, 2, 1) # (batch, channel, seq)

conv_outputs = [conv(embeddings) for conv in self.conv_filters]
concat_outputs = torch.cat([c.squeeze(-1) for c in conv_outputs], dim=1)

logits = self.classifier(concat_outputs)
```

## 학습

### ✓ 손실 함수

- 이진 분류 → BCEWithLogitsLoss

### ✓ 최적화 함수

- Adam (일반적으로 텍스트 CNN에서 가장 잘 작동)

### ✓ 출력

- Train Loss 감소
- Validation Accuracy 증가 (모델 성능 향상)

---

# Text CNN이 좋은 이유

## ✓ 장점

- n-gram 특징을 자동 학습
- 순환구조(RNN) 없이도 문맥적 패턴 인식 가능
- 병렬 처리 빠름 → 대규모 데이터에서 효율적
- 사전 학습된 임베딩과 잘 결합됨

## ✓ 단점

- RNN처럼 장기 의존성(long dependency)에 강하지는 않음
  - 문장 순서 전체 구조를 반영하기는 어려움
-